

PyMongo Cheatsheet

MongoDB with Python — from your first query to sharding · CRUD · Operators · Indexes · Aggregation · Replication · Sharding

`col.verb_one | many ( filter, change )`

verb = insert · find · update · delete | **one** = first match · **many** = all matches | **filter** = which docs · **change** = what to do

1 Connect

```
# pip install pymongo
from pymongo import MongoClient

client = MongoClient(
    "mongodb://localhost:27017")
db = client["shop"]      # database
col = db["users"]        # collection
```

★ **remember** DB & collection are created lazily — they appear on the first insert. No schema needed.

2 A document = a Python dict

```
{
  "_id": ObjectId("..."),  # auto primary key
  "name": "Asha",
  "age": 25,
  "addr": {"city": "Delhi", "pin": "110001"},
  "tags": ["python", "mongo"],
  "joined": datetime(2026, 1, 15),
}
```

3 Insert — create

```
r = col.insert_one({"name": "Asha", "age": 25})
r.inserted_id      # _id of the new doc

r = col.insert_many([
    {"name": "Ravi", "age": 30},
    {"name": "Meena", "age": 27},
])
r.inserted_ids
```

4 Find — read

```
col.find_one({"name": "Asha"})  # dict or None

for doc in col.find({"age": {"$gt": 25}}):
    print(doc)                  # cursor: loop it

# projection = choose fields (1 keep, 0 hide)
col.find({}, {"name": 1, "age": 1, "_id": 0})

# pagination: sort -> skip -> limit
col.find().sort("age", -1).skip(10).limit(5)
```

★ **remember** `find_one` returns one dict; `find` returns a cursor — loop it or wrap in `list()`. Empty filter `{}` means "everything".

5 Update

```
col.update_one(
    {"name": "Asha"},      # filter: which doc
    {"$set": {"age": 26}}, # change: what to do
)
col.update_many({"city": "Delhi"},
    {"$inc": {"score": 10}})

# upsert: update, or insert if nothing matched
col.update_one({"email": em}, {"$set": data},
    upsert=True)

col.replace_one({"name": "Asha"}, new_doc)
r.matched_count; r.modified_count
```

★ **remember** The change must use an operator like `$set` — a plain dict here is an error (only `replace_one` takes a full doc).

6 Delete

```
col.delete_one({"name": "Asha"})
r = col.delete_many({"age": {"$lt": 18}})
r.deleted_count
col.delete_many({})  # empty the collection
col.drop()           # remove whole collection
```

7 Change & get the doc back

```
from pymongo import ReturnDocument
doc = col.find_one_and_update(
    {"name": "Asha"}, {"$inc": {"age": 1}},
    return_document=ReturnDocument.AFTER)
doc = col.find_one_and_delete({"name": "Ravi"})
```

8 Count & distinct

```
col.count_documents({"age": {"$gt": 25}})
col.estimated_document_count()  # whole, fast
col.distinct("city")            # unique values
```

1 Comparison — inside {field: {...}}

```
{ "age": { "$eq": 25 } } # == same as { "age": 25 }
{ "age": { "$ne": 25 } } # !=
{ "age": { "$gt": 25 } } # > $gte >=
{ "age": { "$lt": 25 } } # < $lte <=
{ "city": { "$in": [ "Delhi", "Pune" ] } } # any of
{ "city": { "$nin": [ "Goa" ] } } # none of these
{ "age": { "$gte": 18, "$lte": 30 } } # range
```

2 Logical

```
{ "age": 25, "city": "Delhi" } # comma = AND
{ "$and": [ { ... }, { ... } ] } # explicit AND
{ "$or": [ { "age": { "$lt": 18 } },
            { "vip": True } ] }
{ "age": { "$not": { "$gt": 30 } } } # negate a test
{ "$nor": [ { ... }, { ... } ] } # none may match
```

3 Field checks, null & regex

```
{ "phone": { "$exists": True } } # field present?
{ "age": { "$type": "int" } }
{ "name": { "$regex": "^a", "$options": "i" } }

{ "phone": None } # null OR missing
{ "phone": { "$exists": False } } # truly missing
{ "phone": { "$type": "null" } } # present + null
```

4 \$expr — compare two fields

```
{ "$expr": { "$gt": [ "$spent", "$budget" ] } }
# docs where one field beats another
```

5 Nested docs & arrays (query)

```
# doc: { "addr": { "city": "Delhi",
#               "tags": [ "py", "db" ] } }
{ "addr.city": "Delhi" } # dot = go inside
{ "tags": "py" } # array CONTAINS it
{ "tags": { "$all": [ "py", "db" ] } } # has both
{ "tags": { "$size": 2 } } # exactly 2
# same element passes all tests:
{ "marks": { "$elemMatch":
    { "$gt": 80, "$lt": 90 } } }
```

6 Update operators — the change dict

```
{ "$set": { "age": 26 } } # set / add field
{ "$unset": { "temp": "" } } # remove field
{ "$inc": { "views": 1 } } # add to number
{ "$mul": { "price": 1.1 } } # multiply
{ "$rename": { "nam": "name" } }
{ "$min": { "low": 5 } } # keep smaller
{ "$max": { "high": 9 } } # keep bigger
```

7 Array updates

```
{ "$push": { "tags": "new" } } # append
{ "$push": { "tags": { "$each": [ "a", "b" ] } } }
{ "$addToSet": { "tags": "py" } } # no dupes
{ "$pull": { "tags": "old" } } # remove it
{ "$pop": { "tags": 1 } } # 1 last, -1 first
```

```
# "$" = the array element the filter matched
col.update_one( { "marks": 40 },
    { "$set": { "marks.$": 45 } } )
```

```
# "$[x]" = EVERY element passing a condition
col.update_many( { },
    { "$set": { "marks.$[m]": 40 } },
    array_filters=[ { "m": { "$lt": 40 } } ] )
```

8 Indexes — make queries fast

```
col.create_index("email", unique=True)
col.create_index([ ("city", 1), ("age", -1) ])
col.create_index("createdAt",
    expireAfterSeconds=3600) # TTL
col.create_index([ ("bio", "text") ])
col.find( { "$text": { "$search": "python" } } )
col.list_indexes(); col.drop_index("email_1")
col.find( { "city": "Delhi" } ).explain()
```

★ **remember** Index the fields you filter and sort on most. **1** = ascending, **-1** = descending. TTL indexes auto-delete old docs.

9 Handy patterns

```
# multi-key sort: city A-Z, then age high-low
col.find().sort([ ("city", 1), ("age", -1) ])
# case-insensitive exact match
{ "name": { "$regex": "^asha$", "$options": "i" } }
# between two dates
{ "at": { "$gte": start, "$lt": end } }
```

docs → **\$match** → **\$group** → **\$sort** → **result**
A pipeline is a list of stages. Docs flow through; each stage transforms them and passes them on. **Order matters.**

1 The skeleton

```
result = db.orders.aggregate([
    {"$match": ...}, # stage 1: filter
    {"$group": ...}, # stage 2: summarize
    {"$sort": ...}, # stage 3: order
])
for doc in result:
    print(doc)
```

2 Coming from SQL?

SQL	MongoDB stage
WHERE	<code>\$match</code>
SELECT columns	<code>\$project</code>
GROUP BY	<code>\$group</code>
HAVING	<code>\$match</code> after <code>\$group</code>
JOIN	<code>\$lookup</code>
ORDER BY	<code>\$sort</code>
LIMIT / OFFSET	<code>\$limit</code> / <code>\$skip</code>
COUNT(*)	<code>{"\$sum": 1}</code> or <code>\$count</code>

3 \$group + accumulators

```
{ "$group": {
    "_id": "$city",           # group by field
    "total": {"$sum": "$amount"},
    "count": {"$sum": 1},     # docs per group
    "avg": {"$avg": "$amount"},
    "top": {"$max": "$amount"}, # $min too
    "names": {"$push": "$name"}, # all
    "uniq": {"$addToSet": "$name"}, # unique
}}
```

"_id": None -> ONE group = whole collection

★ **remember** Inside aggregation, "\$field" (with a \$) means "the value of that field".

4 Everyday stages

```
{ "$match": {"status": "paid"} } # filter early!
{ "$project": {"name": 1, "_id": 0,
               "year": {"$year": "$date"}} }
{ "$addFields": {"total":
    {"$multiply": ["$price", "$qty"]}} }
{ "$sort": {"total": -1} }
{ "$limit": 5 }      { "$skip": 5 }
{ "$unwind": "$tags" } # 1 doc per array item
{ "$count": "n" }
{ "$sample": {"size": 5} } # random docs
{ "$out": "report" } # write result to a coll
```

5 \$lookup — join two collections

```
{ "$lookup": {
    "from": "users",           # other collection
    "localField": "user_id",   # field here
    "foreignField": "_id",     # field there
    "as": "user",              # output array field
}}
# then flatten: {"$unwind": "$user"}
```

6 Expression toolbox (in \$project etc.)

```
{ "$year": "$date" } # $month, $dayOfMonth ..
{ "$concat": ["$first", " ", "$last"] }
{ "$toUpper": "$name" } # $toLower
{ "$cond": [{"$gte": ["$avg", 40]},
            "pass", "fail"] } # if-then-else
{ "$ifNull": ["$nick", "guest"] } # default
{ "$multiply": ["$p", "$q"] } # $add, $subtract,
                                # $divide, $round
```

7 Real one: top 3 cities by revenue

```
db.orders.aggregate([
    {"$match": {"status": "paid"}},
    {"$group": {"_id": "$city",
                "rev": {"$sum": "$amount"}}},
    {"$sort": {"rev": -1}},
    {"$limit": 3},
])
```

Replication = copies of the same data (safety, uptime) · **Sharding** = pieces of one big dataset (size, speed)

1 Replication — replica sets

Same data kept on several servers. One **primary** takes all writes; **secondaries** copy it and can serve reads. Gives high availability, not more storage.

PRIMARY 🐞 writes → **SECONDARY** **SECONDARY**

primary dies → automatic election promotes a secondary

```
client = MongoClient(
    "mongodb://h1:27017,h2:27017,h3:27017"
    "?replicaSet=rs0") # finds the primary

# where do reads go?
MongoClient(uri,
    readPreference="secondaryPreferred")
# primary (default) | primaryPreferred |
# secondary | secondaryPreferred | nearest

# how safe are writes?
MongoClient(uri, w="majority") # most must ack
```

one-time setup — mongosh

```
rs.initiate(); rs.add("h2:27017"); rs.status()
```

2 Sharding — horizontal scaling

Splits **one big collection** across servers by a **shard key**. Your code just connects to **mongos** — queries are routed for you.

app → **mongos router** → **shard 1** **shard 2**
shard 3

config servers store which shard holds which key range

setup — mongosh

```
sh.enableSharding("shop")
sh.shardCollection("shop.orders",
    { "user_id": "hashed" })

sh.status()
```

```
MongoClient("mongodb://mongos1:27017") # app
```

★ **remember** Good shard key: many unique values, spreads writes evenly, used in most queries. **"hashed"** = even spread; **{field: 1}** = range queries. Avoid always-increasing keys (timestamps) — all writes hit one shard.

3 Which one do I need?

Replica set: always, in production — it is your safety net.

Sharding: only when one machine can no longer hold the data or the write load. Shards are themselves replica sets.

4 Transactions — all or nothing

```
with client.start_session() as s:
    with s.start_transaction():
        acc.update_one({"_id": "A"},
            {"$inc": {"bal": -100}}, session=s)
        acc.update_one({"_id": "B"},
            {"$inc": {"bal": 100}}, session=s)
# clean exit -> commit, exception -> rollback
# needs a replica set
```

5 Bulk write — one round trip

```
from pymongo import (InsertOne, UpdateOne,
    DeleteOne)

col.bulk_write([
    InsertOne({"x": 1}),
    UpdateOne({"x": 1}, {"$inc": {"x": 1}},
        upsert=True),
    DeleteOne({"x": 99}),
])
```

6 Change streams — live updates

```
for ch in col.watch():
    print(ch["operationType"],
        ch.get("fullDocument"))
# fires on insert/update/delete, in real time
# (needs a replica set)
```

7 Good to know

```
from bson import ObjectId
col.find_one({"_id": ObjectId(id_str)})
```

```
from datetime import datetime, timezone
{"at": datetime.now(timezone.utc)} # UTC!
```

```
from pymongo.errors import DuplicateKeyError
try: col.insert_one(doc)
except DuplicateKeyError: ...
```

```
client.admin.command("ping") # test the conn
# create ONE MongoClient per app (it pools)
```